

official_matrix_88 완전 재현 캠페인 실행 계획서

커널 생성 에이전트 8종 × 공식 벤치마크 8개 = 64셀 매트릭스

2026년 7월 13일

Abstract

본 문서는 official_matrix_88 완전 충실(faithful) 재현 매트릭스(8 에이전트 × 8 벤치마크 = 64 셀)를 실제로 실행하기 위한 계획서다. **선행 인프라 작업은 모두 완료되었다**: (1) 8종 에이전트를 각자의 실제 구현/모델로 재현하여 KernelBench 공식 오라클 판정을 확보하였고, (2) 8개 벤치마크의 공식 오라클을 전부 NVIDIA RTX PRO 6000 (Blackwell, sm_120)에서 배선·검증하였다. 따라서 남은 것은 GPU 실행뿐이며 **즉시 착수 가능하다**. 본 계획서는 GPU 요건, 시간/비용(실측 기반), 예상 결과물과 결론, 실험 설계, 논문 연결 방안을 제시한다. 모든 소요 시간은 본 하드웨어에서 실제로 측정한 값에 근거하며 추정치는 명시한다.

1 배경 및 현재 상태

매트릭스는 서로 다른 오픈소스 커널 생성 에이전트를, 각자의 실제 구현/모델로 구동하여, 각 벤치마크의 공식 오라클로 통일 평가한다. 저장소에 커밋된 기존 88셀은 모든 에이전트가 하나의 제너릭 드라이버로 “smoke용 단순 torch 코드”를 생성한 것으로 에이전트 성능이 아니다.

두 축이 모두 준비 완료되었다:

- **에이전트 축(8종 재현 완료)**: cudaforge, autokernel, autotriton, kernelllm, ksearch, kernelskill, drkernel, incoder32b. 각 드라이버와 서빙 절차 확립. KernelBench(level1/19_ReLU) 공식 판정: **6/8 correct, 2/8 충실한 실패**(kernelllm 차원 하드코딩, incoder32b 커널 구조 오류 — 모델 자체 오류).
- **벤치마크 축(8 오라클 배선 완료)**: 8개 벤치마크의 공식 오라클을 전부 Blackwell에서 실행하여 구조화된 공식 판정 반환을 확인. 러너의 배선은 코드 수정 없이 동작(환경 설정 + prepare만 필요).
- **범위 외 3종**: cuda_ll, cuda_agent(RL 모델 비공개), geak(AMD 전용).

2 GPU 요건

2.1 최소 요건 (실측)

본 재현/배선 전 과정을 **단일 RTX PRO 6000 (Blackwell, 96GB) 한 장**으로 수행하였다. 핵심 제약:

- **VRAM**: incoder32b(32B) 가중치가 **약 60GB**라 대용량 카드가 필요하다. 서빙과 커널 eval이 같은 GPU를 공유하므로, 32B는 “서빙→생성→서버 내림→eval” 순차로 처리하였다.
- **툴체인**: nvcc + 해당 GPU 아키텍처. Blackwell은 TORCH_CUDA_ARCH_LIST=12.0(sm_120). 타 GPU 이전 시 이 값을 변경(H100=9.0, H200=9.0).

옵션	VRAM/장	32B(incoder) 처리	권장도
4× B200	180GB	서빙+eval 동시 가능	🟢 최상 (sm_120 검증)
4× H200	141GB	동시 가능	🟢 좋음
4× H100	80GB	서빙→생성→내림→eval 순차	🟡 저렴, 32B만 느림
1× RTX PRO 6000	96GB	순차(현재 방식)	무료지만 ~2.8× 느림

Table 1: GPU 옵션. 역할 분담: 2장은 에이전트 모델 서빙(vLLM, 모델별 스왑), 2장은 컴파일·오라클 eval 병렬. 병렬효율 약 70% ⇒ 유효 배속 $\approx 2.8\times$.

2.2 4-GPU 캠페인 권장 구성

3 시간 및 비용 (실측 기반)

셀당 시간 = 태스크 수 $T \times (\text{생성} + \text{eval})$. 본 세션에서 실제 측정한 태스크당 평균:

에이전트 유형	생성 방식	태스크당(생성+eval)
반복형 5종(cudaforge, autokernel, ksearch, kernelskill, drkernel)	다라운드 루프/추론	~8분
단발형 2종(autotriton, kernelllm)	1회 생성	~4분
incoder32b (32B)	무거운 로드+생성	~6분 (+모델 스왑)

Table 2: 태스크당 실측 평균. 벤치마크 1개(8 에이전트)당 $\approx 52T$ 분, $\times 8$ 벤치 = $416T$ 분(단일 GPU 직렬).

4-GPU 유효배속($\div 2.8$)을 적용한 총 시간·비용:

T (셀당 태스크)	벽시계(4×GPU)	GPU-시간	4×H100(~\$10/hr)	4×B200(~\$20/hr)
1 (현재 검증 수준)	~2.5시간	~10	~\$25	~\$50
10 (권장 최소)	~25시간	~99	~\$250	~\$500
50	~5.2일	~496	~\$1,240	~\$2,480
전체 태스크셋	~15일	~1,480	~\$3,700	~\$7,400

Table 3: 총 시간·비용. “전체 태스크셋”은 각 벤치마크의 실제 태스크를 전부 도는 경우: prepare로 확인한 태스크 수(kernelbench 249, robust 200, tritonbench_t 166, _g 184, **multikernel 401**, backendbench 15, pareval 60, sol 21) ⇒ 에이전트당 ~1,300개. multikernel(401)이 커서 전체 실행이 비싸므로, 표준 하위집합 사용을 권장한다.

핵심 변화: 일회성 통합 엔지니어링(8종 통합 + 8 오라클 배선)이 **이미 완료**되었으므로, 남은 비용은 위 표의 **GPU 실행비**뿐이며 즉시 착수 가능하다. 로컬 RTX PRO 6000 1장으로 실행하면 전기요금만 들지만 약 2.8배 느리다($T=10 \rightarrow \sim 70$ 시간).

4 예상 결과물

64셀 매트릭스(8 에이전트 $\times$ 8 벤치마크), 셀마다 공식 오라클 지표:

- 정확도: correct_rate, pass@1
- 성능: speedup / fast_p (fast_1.0/1.5/2.0/3.0)
- 원출력 및 verdict JSON 전량 보존(재현성). 전 셀 원본 충실이므로 replay/N-A 각주 불필요.

5 실험 설계와 내릴 수 있는 결론

5.1 설계: 스캐폴딩 효과와 모델 효과의 분리

8종은 백본 기준으로 두 그룹으로 나뉘어, 두 가지 독립 변인을 분리 측정한다.

- **백본 공유 4종**(cudaforge, autokernel, ksearch, kernelskill — 전부 Qwen2.5-Coder-14B): 동일 모델 위에서 **에이전트 스캐폴딩(루프/피드백/메모리/탐색)의 효과**만을 통제 비교.
- **자체 모델 4종**(autotriton 8B, kernelllm 8B, drkernel 14B, incoder32b 32B): 각자의 **훈련된 모델의 효과**를 비교.

5.2 예비 신호($T=1$ 에서 관측) → 확대 시 결론

1. **반복형 에이전트가 단발 모델보다 견고하다.** ReLU 단일 태스크에서도 단발 모델 2종 (kernelllm 차원 하드코딩, incoder32b 커널 구조 오류)이 실패한 반면, 반복형(cudaforge/ksearch/ kernelskill 등)은 통과했다. 특히 ksearch는 1라운드 실패 후 **2라운드에서 자가수정**하여 통과, kernelskill은 메모리 루프로 자가수정하였다 ⇒ “하드웨어 피드백/자가수정 루프의 가치”.
2. **벤치마크 난이도 구조.** pareval(C++ 완성)은 어렵고, backendbench는 커버리지 갭 (add/mul/sub/div 미커버), kb/triton 계열은 상대적으로 쉬움.
3. **패러다임 효과.** 반복형(NCU 피드백/메모리)/단발 모델/RL 모델의 상대 우위를 벤치마크 계열 별로 규명.

5.3 못 내리는 결론(정직한 한계)

geak/AMD 비교, cuda_agent/cuda_11의 라이브 성능(모델 비공개), T 가 작을 때의 세밀한 순위.

6 논문 연결

- **Table 2 (Taxonomy):** 에이전트 분류(라이브/모델-온리/replay) + 재현 가능성(faithful 8 / replay 2 / N-A 1).
- **Contribution — 인프라:** 모든 벤치마크의 공식 오라클을 하나로 배선한 이식형 통합 하네스 + 8종 에이전트 실제 구현 통합, **Blackwell(sm_120)에서 end-to-end 검증 완료.**
- **Table 3 (Main Results):** 64셀 완전 충실 매트릭스(정확도 + fast_p). 논문의 핵심 표.
- **Analysis:** §5.1의 분리 설계에 기반 — 백본 통제 4종으로 “스캐폴딩 효과”, 자체모델 4종으로 “모델 효과”.
- **Limitations / Threats to Validity:** 제외 3종(모델 비공개 2, AMD 1), 백본 통제(공유 4종은 모델이 아닌 루프만 비교), T 규모, 하드웨어(sm_120) 특수성.
- **Reproducibility 부록:** 하드-윈 이슈(arch 핀, stale lock, GPU 공유, flashinfer, PYTHON-PATH/timeout, grid 심, Llama 오버라이드, 오라클 배선 등) + 커맨드.

7 실행 계획(권장)

1. **운영점 선택:** 논문 초판은 $T = 10(4 \times H100$ 기준 ~ 25 시간, $\sim \$250$)로 통계적 최소 의미를 확보. 필요 시 핵심 벤치마크만 T 를 높여 심화.
2. **실행 순서:** 백본 공유 4종(Qwen14B)을 먼저 묶어 실행(모델 스왑 최소화) → 자체 모델 4종을 모델별로 배치(autotriton → kernelllm → drkernel → incoder32b 순, 각 서빙 후 8벤치 일괄).

3. 산출: 64셀 매트릭스 CSV/JSON → `summarize_matrix_88.py`로 표 생성 → 논문 Table 3.
4. 환경: 각 실행에 `TORCH_CUDA_ARCH_LIST`(대상 GPU arch), 실행별 새 `TORCH_EXTENSIONS_DIR`, 오픈라인 플래그, LLM 엔드포인트 설정(`repro_env.sh` 참조).

부록: 준비 완료 체크리스트

- 8종 에이전트 재현 드라이버 + 서빙 절차
- 8개 벤치마크 공식 오라클 배선(Blackwell 검증)
- 태스크 매니페스트 prepare 완료
- 환경 스크립트(`repro_env.sh`) + 하드-원 이슈 문서화
- 재현 보고서(`docs/repro_report.tex` + PDF)

남은 작업 = 위 실행 계획대로 GPU 캠페인 실행($T>1$)뿐.